

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 02 -

MODULARIDADE: FUNÇÕES, CLASSES E PACOTES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	20
O QUE VOCÊ VIU NESTA AULA?	23
REFERÊNCIAS.....	24

LEANDRO

O QUE VEM POR AÍ?

Imagine tentar ajustar uma pequena mudança em um script gigante com centenas de linhas – uma tarefa simples pode virar um pesadelo! Você já se pegou rolando infinitamente um notebook Jupyter ou arquivo .py em busca de onde alterar algo sem quebrar o resto? Nessa situação, qualquer ajuste vira risco, pois tudo está interligado de forma frágil.

Esse cenário descreve um código monolítico: uma massa única e densa, difícil de entender e com muita “dívida técnica” acumulada (ou seja, atalhos de implementação que cobram seu preço mais tarde em manutenção). Felizmente, há solução: modularizar o código. Ao quebrar programas complexos em partes menores e bem definidas, ganhamos clareza e controle.

Nesta aula, vamos explorar como funções, classes e pacotes ajudam a estruturar projetos de Machine Learning de forma organizada e profissional. Você verá através de exemplos práticos como sair do caos de um código espaguete para a elegância de um design modular. Ao final, ajustar um modelo ou adicionar uma funcionalidade será tão tranquilo quanto encaixar peças de LEGO – cada módulo no seu lugar!

HANDS ON

No hands on desta aula, colocamos a modularidade em prática na refatoração de um código típico de Machine Learning. Em vez de um único bloco fazendo tudo – carregar dados, treinar modelo, calcular métricas – mostramos como separar essas tarefas em funções reutilizáveis e classes bem definidas. Assista às videoaulas antes de prosseguir! A seguir, destacamos um exemplo simplificado de refatoração feito no vídeo, enfatizando o princípio DRY (Don't Repeat Yourself) – Não Se Repita.

Suponha que tínhamos código duplicado para calcular o erro médio quadrático (MSE) no conjunto de treino e de teste, como a seguir (abordagem sem modularidade, violando DRY):

```
# Cálculo de erro repetido (exemplo sem DRY)
mse_treino = sum((y_pred_train - y_true_train)**2) / len(y_true_train)
mse_teste  = sum((y_pred_test  - y_true_test )**2) / len(y_true_test )
```

Código-fonte 1 – Trecho de código monolítico com lógica duplicada para cálculo de erro
Fonte: Elaborado pelo autor (2025)

No hands on, refatoramos esse trecho criando uma função genérica para o MSE e reaproveitando-a para ambos os casos – uma aplicação direta do DRY e da modularização via função.

```
# Cálculo de erro refatorado em função (aplicando DRY)
def calcular_mse(y_true, y_pred):
    return sum((y_pred - y_true)**2) / len(y_true)

mse_treino = calcular_mse(y_true_train, y_pred_train)
```

Código-fonte 2 – Versão modulada do cálculo de erro utilizando função reutilizável
Fonte: Elaborado pelo autor (2025)

Note como o código ficou mais enxuto e claro: definimos `calcular_mse` uma vez só e a reutilizamos. Nos vídeos, há outros exemplos, como encapsular etapas de pré-processamento em funções e até definir uma classe `ModeloML` para agrupar dados e métodos do modelo. Depois de assistir, tente aplicar técnicas semelhantes em um projeto seu.

SAIBA MAIS

Vamos nos aprofundar no conceito de modularidade em programação, destrinchando suas facetas técnicas e importância em projetos de ciência de dados. Nesta seção, mergulhamos nos detalhes, com direito a matemática, algoritmos e boas práticas de engenharia de software.

Por que modularizar?

Modularidade significa dividir um programa em partes menores (módulos) com responsabilidades específicas. Essa ideia se baseia em princípios clássicos da engenharia de software, como separação de interesses (separation of concerns) e coesão alta vs. baixo acoplamento. Em termos simples, cada parte do código deve focar em uma tarefa e as interdependências entre partes devem ser mínimas e bem definidas. Isso traz vários benefícios teóricos e práticos:

Maior manutenibilidade: é muito mais fácil entender e dar manutenção em 10 funções de 50 linhas do que em um bloco único de 500 linhas. Estudos em engenharia de software mostram que a complexidade ciclomática (número de caminhos lógicos em um código) tende a crescer exponencialmente com o tamanho de funções e classes, aumentando a chance de bugs. Com o código modular, limitamos a complexidade de cada unidade, reduzindo probabilidade de erros. Em outras palavras, módulos bem projetados diminuem o “espaço de busca” por bugs: se algo dá errado, conseguimos isolar rapidamente em qual função ou classe está o problema.

Reutilização e DRY: módulos permitem reaproveitar código em diferentes contextos. O mantra DRY – Don't Repeat Yourself, cunhado por Hunt e Thomas no clássico *The Pragmatic Programmer*, prega que “cada pedaço de conhecimento deve ter uma representação única no sistema”. Sempre que você notar code duplicado, é um sinal para extrair aquilo para uma função ou classe reutilizável. Isso evita discrepâncias e retrabalho: se a regra de negócio mudar, você altera em um único lugar. No exemplo do hands on, ao criar `calcular_mse`, garantimos que tanto o erro de treino quanto de teste são computados exatamente da mesma forma, evitando inconsistências. Módulos também ajudam na reutilização entre projetos: por exemplo,

you can port a utility function of data normalization of a project of computational vision for another of time series without having to rewrite anything.

Legibilidade e organização lógica: when we break a large problem into subproblems, we apply the algorithmic concept of “divide and conquer”. In practice, each function or class should ideally do one thing well. This reminds the Principle of Single Responsibility (part of the SOLID design principles) – a function/class = a responsibility. Modular code naturally follows this principle. As a result, the flow of the program becomes more comprehensible, we can read the main of a ML script seeing a sequence of calls of the type `carregar_dados()`, `preprocessar()`, `treinar_modelo()`, `avaliar_modelo()`, almost a narrative, in which each function name summarizes a complex task implemented in detail elsewhere. This abstraction improves clarity: whoever reads the main understands what happens without having to dive into each step at that moment. Readability is not just “cosmetic”: clear code tends to have fewer errors and facilitates the entry of new developers and developers in the project.

Facilidade de teste: modules decoupled are ideal for unit tests. You can test each function in isolation, passing controlled inputs and verifying outputs, without having to run the whole system. For example, we can test the `calcular_mse` function alone with different example vectors and check if it returns the correct value. The same applies to classes: if you have a `ProcessadorDeDados` class with a `limpar_outliers()` method, you can test it by feeding the class with a small dataset and checking the result. This culture of testing individual components is fundamental in data science projects, as it guarantees reliability in each step (data cleaning, feature engineering, prediction etc.). Besides this, when a unit test fails, you already know exactly in which module the defect is, speeding up the debugging. In sum, modular code makes quality measurable and verifiable by the arsenal of automated tests.

Colaboração em equipe: in larger projects (think of a ML production pipeline), modularity allows parallel and integrated work. Different developers (as) can be assigned different modules – for example, one focuses on data preparation functions, another on the model class, another on the results visualization module – without stepping on each other's code. This reduces merge conflicts and speeds up development. Como

os módulos possuem interfaces bem definidas (e.g., função `treinar_modelo(dados_treino, hiperparams)` espera receber os dados já limpos de outro módulo), cada pessoa pode construir e até testar o seu componente com dados simulados. Ao integrar, se todos seguirem o contrato de interface, as peças se encaixam. Essa independência também significa que módulos podem ser substituídos ou melhorados sem refatorar todo o sistema, favorecendo evolutividade. Projetos modulares são como times esportivos: cada jogador tem sua posição e função, contribuindo para o objetivo comum com o mínimo de interferência na função do colega.

Escalabilidade e desempenho: à primeira vista, pode parecer que dividir um código em muitas partes traria lentidão (afinal, chamadas extras de função têm um pequeno custo), mas no contexto de ML, isso é desprezível frente ao custo das operações de álgebra linear intensivas. Na verdade, a modularidade pode ajudar a performance de maneiras indiretas:

- Otimização direcionada – ao isolar um gargalo de desempenho em uma função específica, fica fácil focar otimizações ali (por exemplo, trocar um algoritmo $O(n^2)$ por $O(n \log n)$ em uma parte crítica, sem mexer no resto);
- Uso de bibliotecas otimizadas – com funções bem definidas, podemos substituir uma implementação manual por uma chamada de biblioteca altamente otimizada (C/C++, GPU). Ex.: se tivermos `minha_fft(dados)` feita “no braço” em Python, podemos trocar pela `numpy.fft.fft(dados)` sem impactos colaterais no código que usa essa função;
- Paralelismo e distribuição – módulos independentes podem rodar em paralelo em threads ou mesmo distribuídos em cluster. Por exemplo, se o pipeline de ML estiver separado em etapas (extrair características, treinar modelo, validar resultados), nada impede de executá-las simultaneamente em ambientes diferentes, desde que a etapa seguinte espere os dados da anterior. Frameworks de Big Data e processamento distribuído (como Spark, Dask) se beneficiam dessa divisão de tarefas.

Além disso, alguns módulos podem ser movidos para rodar em GPU enquanto outros ficam na CPU. Essa flexibilidade arquitetural só é possível se o código estiver bem seccionado. Por fim, a modularidade ajuda a controlar o uso de memória: funções

têm variáveis locais que são desalocadas ao terminar, evitando acúmulo desnecessário de dados em escopo global. Em pipelines de dados grandes, processar em etapas moduladas permite liberar da memória etapas já concluídas, diminuindo o pico de uso.

Anteriormente, listamos motivos robustos para modularizar. Resumindo, código modular é mais simples, reutilizável, legível, testável, colaborativo e escalável. Parece bom demais? É porque realmente é uma prática consagrada. Vamos agora dissecar como aplicar modularidade usando as construções do Python: funções, classes e pacotes.

Funções: blocos de construção reutilizáveis

As funções são a unidade básica da modularidade. Toda vez que você identifica um conjunto de instruções que realizam uma tarefa lógica isolada dentro do seu código, é um candidato a virar função. Exemplo: normalizar uma feature, calcular uma métrica, aplicar um filtro de imagem etc. Ao encapsular esse comportamento em uma função com nome descritivo, você conquista abstração – quem chama não precisa saber dos detalhes internos – e reutilização – pode chamar em vários lugares ou projetos.

Uma boa função:

- Tem um propósito claro e um nome que o reflita (seguindo a dica “nomeie funções como verbos que deixam claro o que fazem”).
- Recebe entradas (parâmetros) necessárias e retorna um resultado, sem efeitos colaterais inesperados em variáveis globais.
- É pequena. Há um ditado: “uma função deve caber na tela sem rolagem” – não é regra rígida, mas indica que funções muito longas talvez escondam subtarefas que poderiam ser novas funções.
- Lida consistentemente com tipos de entrada e trata erros (quem modulariza também deve pensar em robustez: e se a função receber dados inválidos?).

A modularização por funções também se relaciona com matemática: pense que ao escrever $f(x) = \log(x) + \cos(x)$, estamos definindo um mapeamento para qualquer

x. Em código, definimos `def f(x): return math.log(x) + math.cos(x)`. Depois usamos `f(2)` sabendo exatamente o que esperar. Essa analogia lembra o conceito de função matemática – uma entrada, uma saída – idealmente nossas funções de código deveriam se comportar assim, o que facilita raciocinar sobre elas. Em ciência de dados, é comum termos funções puras que transformam dados: ex. `limpar_outliers(dados)` poderia implementar uma fórmula estatística para substituir outliers (como valores além de 3 desvios-padrão). Com uma função, documentamos explicitamente essa lógica (podemos até colocar fórmulas no docstring, tornando o código autoexplicativo para quem conhece a teoria).

Outro ponto é complexidade algorítmica por função. Ao modularizar, conseguimos atribuir a cada função uma complexidade Big-O e pensar em otimizações localizadas. Por exemplo, se `preprocessar_dados(dados)` é $O(n)$ linear no número de registros, e `treinar_modelo(dados)` é $O(n * p)$ (n registros, p parâmetros), sabemos que o treinamento domina o custo. Essa análise ajudaria a decidir, por exemplo, se vale a pena paralelizar a leitura de dados (provavelmente desnecessário se o treinamento já ocupa bem mais tempo). Assim, a modularização auxilia na análise de desempenho por componentes.

Em Python, funções também carregam o conceito de namespace local – as variáveis definidas dentro de uma função não “vazam” para fora. Isso é desejável: módulos bem feitos encapsulam seus detalhes. Se acidentalmente usamos um nome de variável já existente fora, não há conflito graças ao escopo de função. Essa proteção de namespace é um alicerce para evitar efeitos colaterais confusos.

Em suma, comece a modularizar identificando operações repetitivas ou blocos lógicos claros e “funcionalize-os”. No caso do nosso projeto de ML hipotético: carregamento de arquivos, limpeza de NAs, cálculo de estatísticas descritivas, treinamento do modelo, cálculo de acurácia – cada qual pode ser uma função separada. Você notará o código ficando mais curto na função principal e mais verboso em definição de funções – o que é bom, pois agora temos peças nomeadas e documentadas em vez de uma massa anônima.

Vale mencionar que o Python oferece até ferramentas para facilitar modularização e reutilização de funções, como notebooks compartilháveis ou utilitários. Mas, no fundo, definir boas funções é uma arte que se refina com prática e aplicação consistente do DRY. Na dúvida, lembre: se escreveu algo muito parecido

duas vezes, faça disso uma função e use nos dois lugares. E não tenha medo de funções pequenas – às vezes, vale ter uma única linha em uma função se isso agregar significado (ex.: `def graus_para_radianos(x): return x*math.pi/180` pode ser trivial, mas torna o código que a usa mais legível).

Classes e Orientação a Objetos: estruturas modulares com estado

Se funções são ótimas para ações, classes são ótimas para modelar entidades com estado e comportamento. Em projetos de ML, um exemplo clássico de uso de classes é representar um modelo de machine learning como um objeto. Pense no scikit-learn: você instancia um `LinearRegression()`, depois chama métodos como `.fit(X, y)` e `.predict(X_new)`. Por trás disso está a ideia de encapsular os dados do modelo (pesos treinados, hiperparâmetros) e as operações sobre o modelo dentro de uma classe.

Como isso contribui para modularidade?

Encapsulamento: a classe guarda internamente certas variáveis que só dizem respeito a ela. Por exemplo, depois de `modelo.fit()`, os coeficientes calculados ficam armazenados dentro de `modelo` (tipicamente em `modelo.coef_`). O resto do programa não precisa lidar diretamente com esses detalhes, apenas com a interface pública (`fit`, `predict`). Isso segue o princípio de information hiding de David Parnas (1972): esconder detalhes de implementação, expondo só o necessário.

Assim, reduzimos o acoplamento – outras partes do código interagem com a classe via métodos, mas não dependem de como ela faz as coisas internamente. Se decidirmos mudar a implementação do treino (por exemplo, trocar a fórmula de cálculo de gradiente), nada fora da classe precisa mudar, contanto que métodos `fit` e `predict` continuem funcionando conforme especificado.

Reutilização via herança: a Orientação a Objetos permite criar hierarquias de classes. Uma classe base define atributos e métodos comuns e subclasses especializam comportamentos. Isso é útil para evitar duplicação de código entre classes similares (DRY aplicado a estruturas de dados). Imaginemos que tenhamos classes para diferentes algoritmos de classificação (árvore de decisão, rede neural, SVM etc.). Todas podem herdar de uma classe abstrata `ModeloClassificacao` que

define métodos genéricos como `avaliar_desempenho()` (talvez calculando acurácia, matriz de confusão) e propriedades como `nome_do_modelo`. Cada subclasse implementa seu `treinar()` específico, mas herdará funcionalidades comuns.

Com isso, não repetimos código de avaliação em cada classe – escrevemos uma vez na superclasse. O blog Riverfount (2025) ilustra essa ideia com classes de funcionários vs. gerentes, onde a herança evita duplicar atributos comuns. No contexto de ML, herança e polimorfismo (tratar instâncias de subclasses como da classe pai) permitem, por exemplo, trocar um modelo por outro sem mudar o restante do pipeline, desde que todos sigam a mesma interface base.

Organização por responsabilidade: classes permitem agrupar funções (métodos) relacionadas ao mesmo conceito. Em um projeto de ML, poderíamos ter uma classe `DataLoader` com métodos `ler_csv`, `ler_json`, etc., que cuidam de entrada de dados; uma classe “Preprocessador” com métodos `normalizar`, `codificar_categorias` etc.; e classes de modelo para a parte de aprendizado. Cada uma encapsula um “subconjunto” da lógica do projeto. Isso é coesão: elementos relacionados mantidos juntos.

A vantagem sobre ter apenas funções soltas é na manutenção de estado compartilhado: por exemplo, a classe “Preprocessador” poderia ter um atributo `regras_de_limpeza` carregado no `init` que seus métodos utilizam. Sem classe, teríamos que passar essa estrutura para várias funções manualmente, aumentando assinaturas e chance de erro. Com a classe, estado e operações viajam juntos, de forma consistente.

Lifecycle e gerenciamento de recursos: em contextos mais avançados, classes podem ajudar a controlar recursos de maneira modular. Por exemplo, você pode ter uma classe `GerenciadorGPU` que ao instanciar aloca memória na GPU e ao ser destruída libera (via método `__exit__` de um context manager, por exemplo). Outro caso: uma classe `Experimento` que ao instanciar cria diretórios para logs e ao final do processamento salva resultados e fecha arquivos. Esse padrão de construção e destruição encapsulado é difícil de conseguir com funções avulsas sem espalhar lógica de controle de recursos pelo código. Com classes bem projetadas, você consegue um controle granular de memória e recursos, crucial em projetos que lidam com datasets enormes ou treinamento distribuído.

Uma recomendação importante ao criar classes: garanta que cada classe siga também a responsabilidade única. **Evite classes "Deus"** que fazem de tudo – isso apenas cria um monólito em outro nível. Prefira muitas classes coesas a poucas classes gigantes. Em Python, por ser multiparadigma, não é obrigatório usar classe para tudo – às vezes, funções bastam. Mas sempre que precisar manter um estado ou quiser aproveitar herança/polimorfismo, as classes são suas amigas na modularização.

Um detalhe de Python: diferentemente de linguagens estáticas, não temos modificadores de acesso (public, private) estritos. Mas por convenção usamos prefixo “_” em atributos ou métodos para indicar “uso interno”. Exemplo: na classe `ModeloML`, podemos ter `_inicializar_pesos()` só para uso dentro da classe. Quem usar de fora estará “por sua conta e risco”. Essa convenção ajuda a reforçar o encapsulamento.

Módulos e Pacotes: organização em arquivos e diretórios

Chegamos ao nível **macro da modularidade em Python: módulos e pacotes**. Enquanto funções e classes organizam o código dentro de um arquivo, os módulos organizam em múltiplos arquivos e os pacotes organizam diversos módulos em pastas. Em Python, módulo é um arquivo `.py` contendo definições (funções, classes, variáveis) e código executável opcional. Já pacote é uma pasta contendo módulos (e possivelmente subpastas), geralmente com um arquivo especial `__init__.py` indicando ao Python que ali há um pacote importável.

Por que isso interessa? Para projetos de ML que crescem, logo percebemos que colocar todas as funções e classes em um único arquivo `model.py` fica impraticável. Módulos permitem separar por assunto: por exemplo, criar um `dados.py` para funções de data loading e limpeza, um `modelo.py` para classes de modelo, um `avaliacao.py` para funções de métrica e assim por diante. Dessa forma, cada arquivo se torna um contexto isolado e lógico – exatamente como capítulos de um livro técnico. Inclusive, podemos ter múltiplos colaboradores(as) editando módulos diferentes simultaneamente sem conflitar.

A linguagem Python incentiva isso através do comando `import`. Uma vez que criamos nossos módulos, podemos usá-los em outros, importando suas funções/classes conforme necessário. Ex.: no `main.py`, podemos fazer `from dados`

`import carregar_csv`, `limpar_outliers` e `from modelo import ModeloML`. Assim, o main orquestra chamadas entre módulos, mas a implementação de cada parte está em seu arquivo próprio. Essa separação melhora até a **compilação mental do programa**: se surgir um bug em uma métrica calculada errada, você sabe que possivelmente o erro está no módulo `avaliacao.py` e nem precisa abrir os outros arquivos para investigar. Os módulos atuam como **fronteiras** dentro do código.

Pacotes entram em cena quando o projeto fica ainda maior ou quando queremos distribuir um conjunto de módulos de forma organizada (como uma biblioteca interna ou externa). Agrupar módulos relacionados em um pacote ajuda na hierarquia de nomes (namespace hierarchy). Por exemplo, poderíamos ter um pacote `meu_projeto_ml`: dentro dele, um subpacote `preprocessamento` com módulos `limpeza.py`, `transformacao.py`; um subpacote `modelos` com `regressao.py`, `redeneural.py`; e assim por diante. Essa estrutura hierárquica evita colisão de nomes (posso ter funções `carregar()` tanto em `preprocessamento.dados` quanto em `modelos.hiperparametros`, sem conflito, pois os namespaces são distintos). Além disso, facilita usar somente partes necessárias: se outro projeto quiser aproveitar apenas sua rotina de pré-processamento, pode importar só aquele pacote específico, sem puxar dependências desnecessárias.

Criar um pacote Python é simples, basta estruturar diretórios e incluir `__init__.py`. O `__init__.py` pode ficar vazio ou inicializar algo do pacote. Depois, conseguimos importar submódulos com sintaxe `dot`. Por exemplo: `import meu_projeto_ml.preprocessamento.limpeza` as `limpeza` e então usar `limpeza.limpar_outliers(dados)`. Essa modularização física do código também é fundamental para publicar “SDKs” ou bibliotecas internas.

Suponha que sua equipe de dados desenvolveu um conjunto de funções e classes úteis e quer que outras equipes usem. Empacotar isso em um pacote interno (com `setup.py`/`pyproject.toml` para instalar via `pip`) é uma ótima estratégia de compartilhamento. Só se consegue fazer isso adequadamente se o código for modular – ninguém quer importar uma biblioteca confusa em que funções não estão organizadas.

Um ponto importante é planejar a partição lógica do projeto em módulos/pacotes. Não há bala de prata, mas algumas dicas:

- Agrupe por funcionalidade, não por tipo. Ex.: dados.py contendo funções de leitura de dados de vários formatos é melhor do que csv_utils.py, json_utils.py separados – a não ser que sejam muitos a ponto de justificar subagrupar. Pense: quem vai usar provavelmente quer “tudo relacionado a dados”. Já separar dados de modelo de visualização faz sentido, pois são preocupações diferentes.
- Evite módulos extremamente pequenos: se um módulo tem apenas uma função, pergunte se é necessário ou se aquela função poderia residir em outro módulo afim. Módulos em excesso prejudicam ao invés de ajudar (muito “vai-e-vem” de arquivos). Tente encontrar um equilíbrio granulado.
- Use pacotes para contextos maiores ou distribuição. Se seu projeto inteiro é específico e pequeno, pode nem precisar de subpacotes. Porém, se está desenvolvendo algo com intuito de reutilização ampla, estruturar já em pacotes com nomes claros (por exemplo myml.data, myml.model etc.) torna mais fácil extrair isso para uma biblioteca independente depois.

Comparativo: Monólito vs. Modular

Vamos consolidar a diferença entre um design monolítico e um modular em alguns aspectos-chave para fixar bem as vantagens discutidas:

ASPECTO	CÓDIGO MONOLÍTICO (NÃO MODULAR)	CÓDIGO MODULAR (FUNÇÕES/CLASSES/PACOTES)
Reutilização de código	Baixa – lógica frequentemente duplicada em diferentes partes. Uma mudança exige editar vários trechos.	Alta – funcionalidades comuns centralizadas em funções ou classes reutilizáveis (princípio DRY). Atualizações ocorrem em um ponto único.
Manutenibilidade	Difícil – base de código intrincada, entender ou alterar algo exige considerar o sistema todo (risco de "mudar A quebrar B").	Elevada – componentes isolados facilitam localizar e corrigir bugs. Cada módulo pode ser compreendido individualmente, reduzindo a carga cognitiva.
Testabilidade	Limitada – testar componentes isoladamente é complicado; tendência a depender de testes integrais mais lentos e menos precisos em identificar falhas.	Ampla – funções e classes podem ser testadas isoladamente (testes unitários). Bugs são pegos depressa e em sua origem específica.
Evolução / Escalabilidade	Engessada – adicionar nova funcionalidade ou adaptar código existente envolve mexer na estrutura toda, com alto risco de efeito cascata. Pouco adequado para ampliações grandes.	Flexível – novas features podem ser integradas criando novos módulos ou estendendo classes, afetando minimamente o restante. O sistema escala para maior complexidade de forma organizada (inclusive permitindo múltiplos colaboradores).

Desempenho e recursos	Potencialmente ineficiente – otimizações localizadas são difíceis. Uso de recursos pode ser subótimo (e.g., variáveis globais ficam ocupando memória).	Otimizável – gargalos de performance são identificáveis por módulo e tratados isoladamente. Facilita uso de soluções paralelas/distribuídas e liberação de recursos ao término de cada componente.
Organização do código	Caótica – tudo no mesmo lugar, alto acoplamento. Difícil reutilizar parcialmente ou compartilhar partes.	Estruturada – lógica de negócio separada em camadas/modos (ex.: dados, lógica, apresentação). Possível reaproveitar módulos ou publicar pacotes para uso externo.

Tabela 1 – Comparação entre código monolítico e código modular

Fonte: Elaborado pelo autor (2025)

Como visto, os ganhos de modularidade permeiam desde aspectos humanos (leitura, colaboração) até técnicos (performance, escalabilidade). Vale lembrar: módulos bem pensados têm alta coesão (elementos fortemente relacionados dentro dele) e baixo acoplamento (pouca dependência intensa com outros módulos) – esses são pilares do design de sistemas robustos.

Refatorando um código monolítico em componentes modulares

Ok, teoricamente tudo soa ótimo. Mas como ir de um scriptzone bagunçado para um projeto modular? Esse processo de transformação é conhecido como refatoração. Refatorar é alterar a estrutura interna do código sem modificar seu comportamento externo. Ou seja, o programa faz as mesmas coisas de antes, porém está organizado de outro jeito (mais limpo). A refatoração deve ser feita de forma incremental e cuidadosa, preferencialmente com apoio de testes para garantir que nada quebre no caminho.

Um plano de ataque comum para refatorar um código monolítico em módulos é:

1. **Identificar seções lógicas** no código grande. Por exemplo, em um típico notebook de ciência de dados podemos ver células ou blocos que: carregam dados, fazem limpeza, treinam modelo, geram gráficos. Cada bloco desses provavelmente vira um candidato a função ou conjunto de funções.
2. **Extrair funções**: copie o código de uma dessas seções para uma nova função. Substitua o trecho original por uma chamada à função. Execute os testes (ou pequenas verificações manuais com entradas conhecidas) para confirmar que o comportamento permanece igual. Ferramentas como VSCode e PyCharm têm recursos de “Extract Method” que automatizam parte disso, minimizando erros.
3. **Introduzir classes, se apropriado**: caso note várias funções atuando sobre uma mesma estrutura de dados, avalie transformá-las em métodos de uma classe que encapsule aquela estrutura. Exemplo: se você tem funções soltas `adicionar_dado(dataset, entry)`, `remover_outlier(dataset)` etc., considere uma classe `Dataset` com esses métodos – especialmente se for preciso manter algum estado (como um log de operações ou parâmetros de limpeza). Use a refatoração “Extrair Classe” se sua IDE oferecer.
4. **Organizar arquivos (módulos)**: quando o arquivo único já estiver mais segmentado em funções e classes, divida-o em arquivos distintos por tema. Crie um novo arquivo, mova algumas definições para lá e ajuste imports onde necessário. Novamente, teste após cada movimentação para garantir que os imports estão corretos e nada quebrou. Exemplo: mova todas as funções utilitárias genéricas para um `utils.py`; coloque classes de modelo em `modelo.py`; funções de avaliação em `avaliacao.py`.
5. **Criar um pacote (opcional)**: se a refatoração for grande ou se planeja reutilizar o código em outros projetos, estruture uma pasta como pacote, distribuindo os módulos dentro. Verifique se os imports internos estão funcionando usando sintaxe relativa ou absoluta conforme conveniência. Às vezes, para não mudar muitos imports de uma vez, você pode criar um `__init__.py` centralizando importações (facilitando quem usa o pacote, por

exemplo, podendo fazer `from meu_projeto import carregar_dados` em vez de `from meu_projeto.dados import carregar_dados`).

6. **Remover redundâncias e dead code:** a refatoração modular é uma ótima oportunidade para eliminar código morto (trechos não usados mais) e unificar trechos duplicados. Lembre-se: após criar uma função para algo repetido, elimine as duplicatas que ficaram obsoletas. Isso reduz drasticamente o tamanho e complexidade global. Desenvolvedores(as) experientes relatam que, frequentemente, uma refatoração bem feita reduz centenas de linhas sem perda de funcionalidade, apenas enxugando repetições e simplificando construções.
7. **Revalidar e documentar:** com tudo reorganizado, rode o pipeline completo em alguns cenários típicos para garantir que o resultado final bate com o original. Depois, aproveite para documentar cada módulo/função/classe com docstrings explicando sua responsabilidade – isso solidifica o contrato modular. Documentação é parte importante da modularidade: cada módulo deve “se explicar” isoladamente.

É normal que durante a refatoração surjam pequenos bugs. Por isso, faça em etapas e use controle de versão para comparar saídas antes/depois. Ferramentas de comparação de resultados ou testes unitários automatizados dão confiança. E lembre-se: refatoração não é um evento único, mas um processo contínuo. Adote uma cultura de ir melhorando o código continuamente, não só quando “a casa já está pegando fogo” em dívidas técnicas.

Uma técnica útil é aplicar refatorações específicas conhecidas: Martin Fowler cataloga dezenas delas em seu livro *Refactoring*. Algumas relevantes aqui: *Extract Function*, *Extract Class*, *Rename Method* (dar nomes melhores deixa o módulo mais claro), *Move Function/Field* (entre classes ou módulos quando faz mais sentido em outro lugar), *Encapsulate Field* (mudar variáveis globais para privadas dentro de classes), entre outras. Cada pequena transformação preserva o funcionamento, mas gradualmente evolui a estrutura para melhor forma.

Em suma, refatorar para modularizar exige disciplina, mas traz enorme retorno: o código fica limpo, pronto para crescer e ser mantido por você e pela equipe. Um código bem modularizado hoje é menos dor de cabeça amanhã – é como pagar juros

agora para não ter dívidas de manutenção maiores depois. Falando nisso, vamos discutir a realidade de mercado: o **débito técnico** em projetos mal estruturados e como a modularidade é a chave para evitá-lo.

EMENDAS

MERCADO, CASES E TENDÊNCIAS

Historicamente, projetos de software que ignoram a modularidade acabam acumulando “dívida técnica”. Essa dívida se manifesta como retrabalho, necessidade de patches emergenciais e até falhas catastróficas. No mercado de tecnologia, há casos emblemáticos que sublinham a importância de código modular e bem projetado.

Knight Capital (2012): uma das histórias mais famosas de falha de software na indústria financeira. A Knight Capital, empresa de trading de alta frequência, perdeu **US\$465 milhões em 45 minutos** e quase foi à falência devido a um bug simples – um trecho de código legado que não foi atualizado em todos os servidores. Basicamente, uma aplicação monolítica de trading teve parte do código replicada em vários módulos (violando DRY) e, em uma atualização, esqueceram de aplicar a mudança em um dos oito servidores. O servidor com código antigo começou a executar ordens erradas em loop. Esse incidente ilustra bem como “repetição de código = multiplicação de riscos”.

Se o sistema tivesse sido projetado para centralizar a lógica crítica em um único módulo (ou se a refatoração tivesse removido o módulo antigo), o erro não teria acontecido ou teria sido facilmente contido. Após o desastre, a lição absorvida no setor foi investir em arquitetura robusta e controles automatizados de implantação – essencialmente, evitar sistemas tão complexos e ad-hoc que pequenas mudanças sejam propensas a erro humano. Ou seja, modularidade e automação andam juntas para prevenir que um detalhe derrube todo o castelo de cartas.

“Hidden Technical Debt in Machine Learning Systems” (Google, 2015): um artigo influente de pesquisadores do Google destacou que em sistemas de ML de produção, **o código do modelo em si é só ~5% do sistema total**, enquanto ~95% é “código cola” e infraestrutura circundante. Esse “código cola” inclui scripts de dados, pipelines de featurização, configuração, monitoramento etc., que se não forem bem estruturados, viram uma “selva de pipelines” de difícil manutenção. Em outras palavras, a falta de modularização adequada em projetos de ML leva a sistemas onde tudo é interdependente de forma confusa – altere uma coisinha e efeitos cascata inesperados surgem (entanglement).

O panorama descrito em 2015 permanece relevante: muitas empresas relataram que, sem práticas de engenharia de software sólidas, projetos de IA emperram ao escalar para produção. A tendência atual de MLOps (Machine Learning Operations) justamente vem para atacar esse problema, trazendo técnicas de engenharia de software para o ciclo de vida de ML. Isso inclui empacotar code em serviços modulares, versionar dados e modelos e criar pipelines declarativos (com ferramentas tipo Kubeflow Pipelines, Apache Airflow, MLflow). Em resumo, o mercado percebeu que um modelo de IA sem uma base modular confiável raramente gera valor contínuo – ele se torna um “experimento de laboratório” difícil de reproduzir. Assim, empresas líderes hoje buscam engenheiros(as) de machine learning com habilidades de arquitetura de software para evitar armadilhas de código instável.

Frameworks e ferramentas modulares em ascensão: outra evidência da relevância do tema é o surgimento de frameworks que impõem modularidade em projetos de ciência de dados. Um exemplo é o Kedro, um framework open-source da McKinsey para criar pipelines de ML estruturados. Ele força você a organizar o projeto em pastas bem definidas (dados brutos, dados processados, modelos, notebooks) e cada etapa do pipeline é um nó modular conectável. Empresas adotaram o Kedro para melhorar a reprodutibilidade e manutenção de projetos – exatamente os ganhos da modularidade.

Em um artigo de 2023, John Adejo demonstra como construir um pipeline modular para detecção de fraude e enfatiza que “código modular, manutenível e reproduzível” é o caminho para colocar modelos em produção de forma eficaz. Outra ferramenta é o Cookiecutter Data Science, que fornece um template de diretórios para projetos de DS, incentivando desde o início a separação de componentes. No desenvolvimento de software convencional, podemos citar a arquitetura de microsserviços como análoga macro da modularidade: sistemas quebrados em serviços independentes comunicando-se via APIs.

Embora microsserviços atuem em nível de sistema distribuído, a filosofia é a mesma – componentes focados e desacoplados facilitam escalar e manter. Em aplicações web modernas, a modularidade vai além do código backend: frontend frameworks também pregam organização modular (por exemplo, separar componentes em aplicações React ou Angular). Ou seja, a indústria de TI como um

todo caminha para estruturas mais modulares em todas as camadas, porque comprovadamente isso traz agilidade e robustez.

Cultura de código limpo e treinamento: empresas como Google, Microsoft, Amazon investem em treinamentos internos de Clean Code, Refactoring Kata etc., para difundir boas práticas como DRY, design modular e padrões de projeto. A ideia é evitar a queda na produtividade conforme o projeto cresce (o efeito bola de neve de código mal organizado). Há também um movimento open de compartilhar esse conhecimento: por exemplo, comunidades no DEV.to e Medium repletas de artigos sobre escrever código melhor e canais no YouTube abordando refatoração (como o Dev Eficiente, Código Simples em português).

A tendência é clara: a pessoa desenvolvedora de dados moderna precisa ir além de saber modelagem estatística – deve ser capaz de construir software de qualidade. Publicações recentes enfatizam isso: a fronteira entre data science e software engineering está diminuindo. Ferramentas automatizadas (até mesmo assistentes de código com IA) podem ajudar a encontrar duplicações e sugerir refatorações, mas o conceito e decisão final vêm do(a) desenvolvedor(a). Portanto, entender e aplicar modularidade é uma skill valorizada. Profissionais que dominam esses conceitos entregam projetos mais confiáveis e colaborativos, o que se traduz em vantagem competitiva para as empresas.

Concluindo esta seção: os casos de fracasso servem de alerta do que não fazer, enquanto as tendências e ferramentas mostram o caminho das pedras para projetos de sucesso. Módulos, funções, classes e pacotes formam a espinha dorsal de sistemas de software robustos, incluindo no mundo de ML/IA. A máxima “escreva código para que outras pessoas entendam, não apenas para a máquina executar” nunca foi tão importante – e modularizar é uma das melhores maneiras de atingir esse objetivo.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu como a modularidade eleva o patamar de qualidade em projetos de programação, especialmente em Machine Learning. Começamos discutindo o conceito e os problemas do código monolítico, depois exploramos funções e classes como ferramentas para quebrar programas em componentes menores, seguindo princípios como DRY e responsabilidade única. Vimos que organizar o código em módulos e pacotes torna mais fácil reutilizar e compartilhar funcionalidades, além de estruturar projetos complexos de forma coerente.

No hands on, refatoramos um código exemplo, mostrando na prática os ganhos imediatos de clareza e reutilização. No saiba mais, aprofundamos aspectos técnicos: do impacto em desempenho (complexidade, paralelismo) à facilidade de testes e colaboração em equipe que uma arquitetura modular proporciona. Também analisamos casos do mundo real: identificamos como a falta de modularidade pode provocar desastres (como no caso Knight Capital) e como a indústria está atenta a isso – adotando frameworks, padrões e uma cultura de código limpo para construir sistemas escaláveis e confiáveis.

Em resumo, você deve agora ser capaz de: organizar código em funções reutilizáveis, projetar classes bem definidas para encapsular lógica e estado e estruturar projetos Python em módulos/pacotes. Mais importante, você compreende por que fazer isso – menos bugs, mais facilidade de dar manutenção, possibilidade de crescer o projeto sem caos e sucesso na colaboração.

REFERÊNCIAS

DIDÁTICA TECH. **O que são módulos e pacotes em Python e como usar?** 2024. Disponível em: <https://didatica.tech/o-que-sao-modulos-e-pacotes-em-python-e-como-usar/> . Acesso em: 19 jan. 2026.

PARNAS, D. **On the criteria to be used in decomposing systems into modules.** 1972. Disponível em: <https://dl.acm.org/doi/10.1145/361598.361623> . Acesso em: 19 jan. 2026.

PEIXINHO, Y. **Princípio DRY (Don't Repeat Yourself).** 2023.. Disponível em: <https://dev.to/yuripeixinho/principio-dry-dont-repeat-yourself-24d7> . Acesso em: 19 jan. 2026.

RIVERFOUNT **DRY**: o princípio que separa código amador de código profissional. 2025. Disponível em: <https://bolha.blog/riverfount/dry-o-principio-que-separa-codigo-amador-de-codigo-profissional> . Acesso em: 19 jan. 2026.

RODRIGUES, G. **Compreendendo Módulos e Pacotes em Python**: Um Guia Completo. 2024. Disponível em: <https://www.linkedin.com/pulse/compreendendo-m%C3%B3dulos-e-pacotes-em-python-guilherme-bueno-rodrigues-inxxc> . Acesso em: 19 jan. 2026.

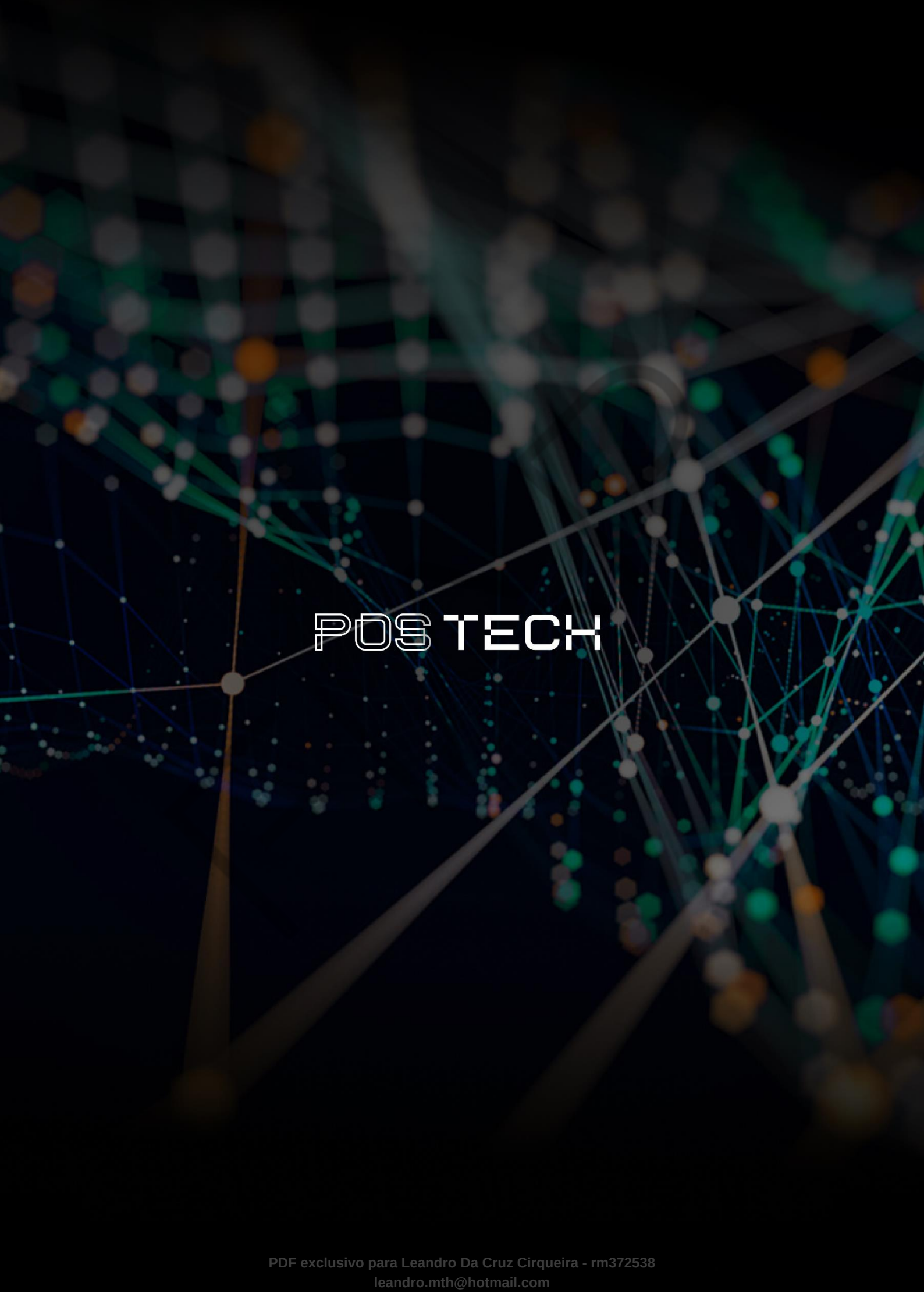
SANTOS, A. S. **Refatoração de Código**: Melhores Práticas para Escrever Código Limpo e Eficiente. 2024.. Disponível em: <https://dev.to/andersoncode66/refatoracao-de-codigo-melhores-praticas-para-escrever-codigo-limpo-e-eficiente-1f80> . Acesso em: 19 jan. 2026.

WANI, M. **The Hidden Technical Debt in Machine Learning**: Why Your ML System Might Be a Ticking Time Bomb. 2025. Disponível em: <https://www.linkedin.com/pulse/hidden-technical-debt-machine-learning-why-your-ml-system-mohit-wani-vzsnf> . Acesso em: 19 jan. 2026.

PALAVRAS-CHAVE

Modularidade de Software. Refatoração de Código. Reutilização de Código.

EMSE



POSTECH